

Smart Home Environment Management

Yoconda Born, Martin Koutenský,
Nicholas Hettiarachchige Don, Aloka Senevirante

19.12.2024

1 Introduction

In our current times, we are trying to be more economical with our resources. *Have I turned off my lights before leaving my apartment? Is it necessary to leave the radiator on all day if no one is home?* But humans are also lazy. If the room is slightly too hot or too cold, many would rather sit down than stand up and change the radiator settings. Sometimes, when we are in a rush, some forget to turn off the lights, and they keep burning until someone returns home to turn them off.

How can you mitigate this behavior? The solution is to have a smart system that turns the lights on and off and adjusts the heating system according to human needs.

In this project, we show a smart home system for a single person living environment that focuses on the heating specifically. In a real world scenario our application area would cover the entire housing area. For our application we have two separate parts. One would be stationed at the entrance doors, detecting whether the human leaves or enters the premises, while the other part communicates with the AC-unit. As it is mandatory we implement the sensor data collection, data processing and as for the Output control, we (hypothetically) manipulate the temperature of the heating system. As for the functionalities that we could choose freely, we agreed on implementing wireless connectivity using MQTT and an Android mobile app utilizing MQTT brokers.

2 Architecture

The architecture consists of the sensor layer, the networking layer, the data management layer and the application layer

1. Sensor layer

This layer contains the main input peripherals. They are split into two groups — one responsible for detecting people entering and leaving, the other for measuring the temperature.

2. Networking layer

For the purpose of this project most services were not deployed, but hosted locally. Nevertheless, communication was established using tokens so no functionality was omitted. The Raspberry Pi Pico W has wired connection to the sensors and publishes the collected data to the HiveMQ database wirelessly, while other data is pulled from subscribed topics to the HiveMQ.

The locally hosted Node Red acts as a computational unit to relieve the microcontroller of this duty. For this purpose it also publishes and subscribes to the HiveMQ database.

3. Data management layer

Data is stored in a serverless HiveMQ database. It can be accessed using tokens and user credentials by various clients. We use Influxdb to better visualize and understand the data. Data is processed in the aforementioned Node Red computational unit.

4. Application Layer

All the information that is being processed and passed through the Node Red and the Raspberry Pi Pico's are being routed through the HiveMQ database through a publisher-subscriber model. The mobile application that was created subscribes to the relevant topics and captures that information for real time view in the mobile application. In addition to that, the mobile application is capable, for publishing messages to the databases and control the output of the peripherals via the application.

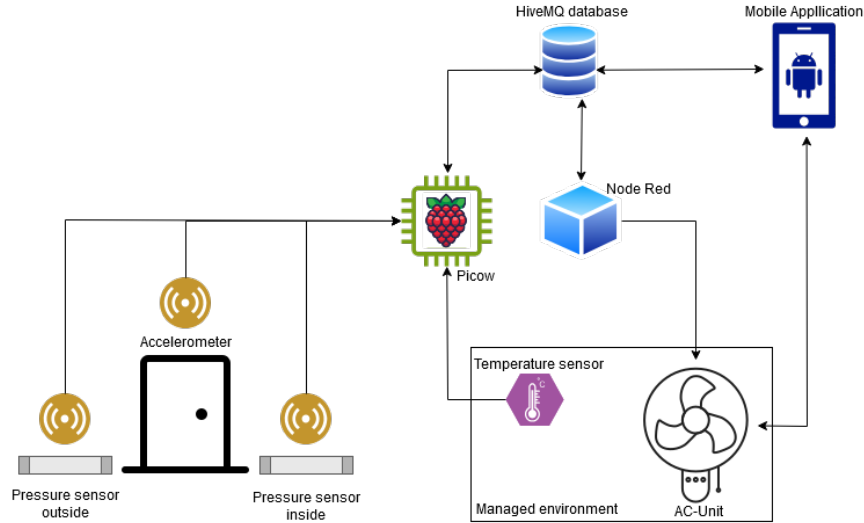


Figure 1: The system architecture

3 Methods & Tools

First we will list the components of this project and then describe in detail their individual use.

- **Microcontroller:**
 - Raspberry Pi PicoW
 - 2x BMP280 (pressure and temperature sensor)
 - ISM330DLC (accelerometer and gyroscope sensor)
- **Libraries:**
 - `microcontroller-umqtt.simple` — MQTTClient for communication with MQTT database
 - `bmp280` — specialized library for using BMP280 sensors
 - `machine` — general micropython library; also used for communication with ISM330DLC (easier than using specialized library because it has too many dependencies)
 - `utime`, `ssl`, `network`, `socket`, `umqtt` `libraries` — helper libraries for network communication
- **HiveMQ**
- **Node Red Flows**
- **Influxdb**
- **Mobile application**

The components are used in the following ways.

In an actual smart home environment the pressure sensors would be placed in front of each side of the main entrance and the accelerometer is attached to the main door. For demonstration purposes and because we only had an atmospheric pressure sensor instead of a "walk-on-pressure-sensor" at hand, we enclosed the pressure sensors in a zip-lock-bag each and simulate the pressure change when a human would usually step on the pressure sensor. An example process of a person leaving their house would be simulated as follows. Squeeze the zip-log-bag, flick the accelerometer and then squeeze the zip-lock-back of the other sensor. The accelerometer's purpose is to confirm the usage of the doors - an indisputable evidence - while the usage of both pressure sensors tell us the direction of the person's movement. The 2 pressure sensors work independently of each other in transmitting data to the HiveMQ server where along with its' reading and that of the accelerometer reading, a decision is made as to if a person is entering or leaving the apartment.

In more detail, we use a sliding window for the measurements. The activation of the pressure sensors is monitored as a deviation from standard values. That

is, if the maximal measurement in the sliding window is more than epsilon higher than the mean measurement, we consider the sensor activated.

$$sensor_activated := \max(window) > \text{mean}(window) + \epsilon$$

For the accelerometer, the expected mean value is 0. Because of this, we only compare the maximum of the absolute value of the measurements with the threshold. This is possible because the accelerometer reports only movement/stillness with error.

$$sensor_activated := \max(abs(window)) > 0 + threshold$$

The sliding window's length is up for discussion and testing. For the moment, we are using 11s long window with 2s intervals at reading sensor values. This allows us to have at least five values to work with at any given moment. The current latency is almost exclusively and linearly dependent on the sliding window's length and the interval of reading. We recognize this downside for the automated part of the project, however, the slowest element of the equation is the heating and cooling down itself. Because of this, we consider this latency acceptable.

The temperature sensors would be placed in the main managed area, but due to hardware limitations they share the same board, and thus location, as the pressure sensors. Data collected by these sensors is utilized in order to control the AC-unit output.

We are using HiveMQ serverless database service in a publish/subscribe mode. The microcontroller publishes sensor readings and subscribes to the Node Red computational unit responses.

The topics are:

- `entrance`
- `pressure1, pressure2`
- `accelerometer`
- `temperature`
- `temp_control`

Topic `entrance` is used when someone either enters or leaves the premises.

Topic `temp_control` is used instead of calling the smart AC unit's API. We consider this a mocked API call that allows us to log it has been done.

Node Red is utilized to better format the sensor readings and to send them to Influxdb for further manipulation. Node Red is running on a separate computational unit which allows us to relieve the microcontroller of the more complex calculations. An important flow is the one dealing with the main entrance because, for our implementation, we need to evaluate readings of three individual sensors over a period of time.

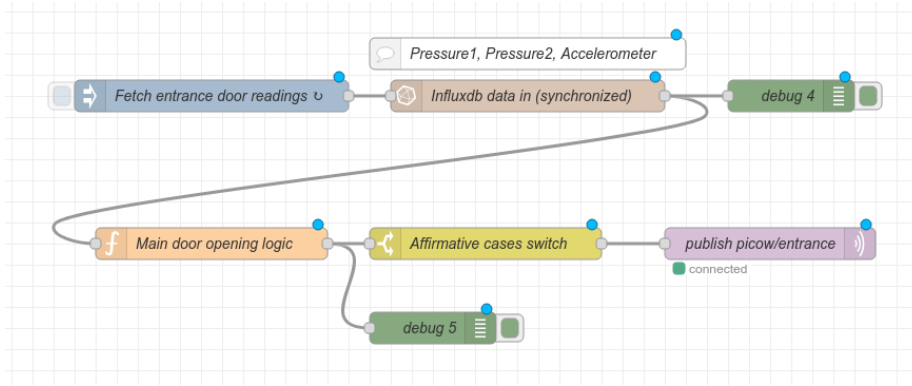


Figure 2: Node Red flow of main entrance

The second important flow deals with evaluating the temperature readings and deciding when to send an automated message to the smart AC unit. This flow includes a timer (a minimum interval) between two automated messages are sent. This will provide enough time for the environment to heat up / cool down and the sensors to measure the change.

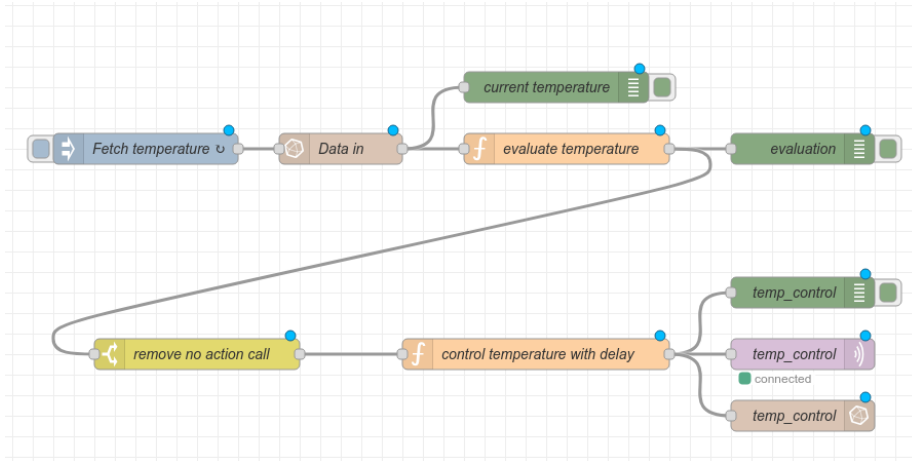
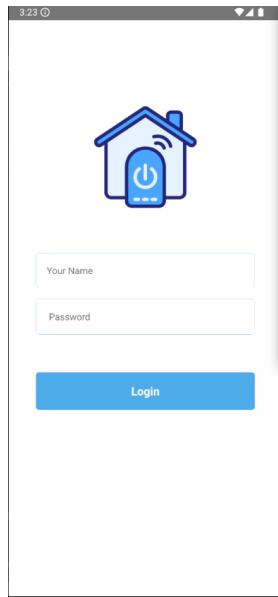


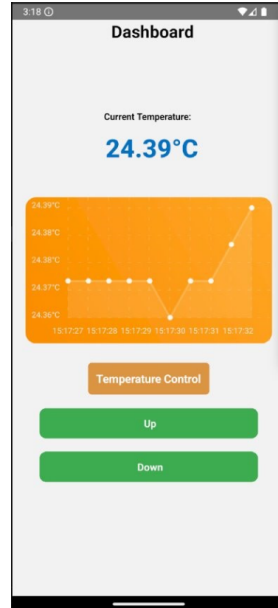
Figure 3: Node Red flow of automated temperature control

The Influxdb allows us to visualize and manipulate data sent from Node Red from HiveMQ. Notable mentions are synchronizing readings from the two pressure sensors and the accelerometer to create tuples, even if the readings were not taken in *precisely* the same time. Such messages are sent at regular intervals and further processed by the Node Red computational unit.

The Smart Home app has the following functions: monitoring temperature,



(a) Login Page



(b) Dashboard page

Figure 4: The view of the mobile application screens

viewing detailed data, and controlling devices via HiveMQ MQTT. The Android application is a React Native-based platform designed for real-time temperature monitoring and trend visualization. Key features include MQTT integration via HiveMQ, utilizing the `mqtt.js` library configured in `mqttClient.js` to enable seamless communication with the broker. The application incorporates data visualization, leveraging the `react-native-chart-kit` library to display temperature trends graphically on the dashboard. State management is handled modularly, using React's built-in `useState` and `useEffect` hooks to manage MQTT subscriptions and ensure responsive data updates. App includes two screens - Login and Dashboard.

- LoginScreen: Handles user authentication.
- DashboardScreen: Show detailed visualizations for specific metrics and displays temperature data.

4 Evaluation

Our project has several implemented parts and several mocked parts. For us to be able to test something properly, we need to restrain ourselves to the fully implemented parts. One of these is the main entrance system. An important

aspect of the main entrance system is the frequency of messages, which will affect both the energy consumption and the quality of the readings. Since our sensors are on a steady power supply we cannot measure the energy consumption. What we can do is find the lowest frequency that yields satisfactory results. We can safely assume this will lower the power consumption as much as possible.

- We will compare reading entrance values every 2 seconds as opposed to every 10 seconds.
- We will compare evaluating entrance values every 10 seconds as opposed to every 5 minutes.

5 Results

Conclusions were gathered empirically through trial and error while fine tuning size of the sliding windows and the frequency of the readings. A representative segment was found and visualized to showcase the reported phenomenons.

- Please refer to figure 5. Reading values only every 10s is not often enough. Both the pressure and the accelerometer have time to stabilize again and thus a person's entrance/leave can be completely missed. Original intervals of 2s were almost ideal. With intervals of 3s the readings were clear enough to use without noticeable mistakes. This could be further improved with more controlled gathering methods - accelerometer on a lever and pressure sensors in a reusable tool rather than a loose zip-lock-bag.
- Please refer to figure 6. Evaluating data only every 5min can create a problem, where multiple events can be registered happening, but our simple algorithm mistakes which actions belong to which event, resulting either in:
 - misinterpreting of the event (evaluating entering as leaving and vice versa);
 - not evaluating an event at all.

Using an interval of 20s seemed short enough to not allow for this to happen, yet longer than 10s and thus using fewer resources.

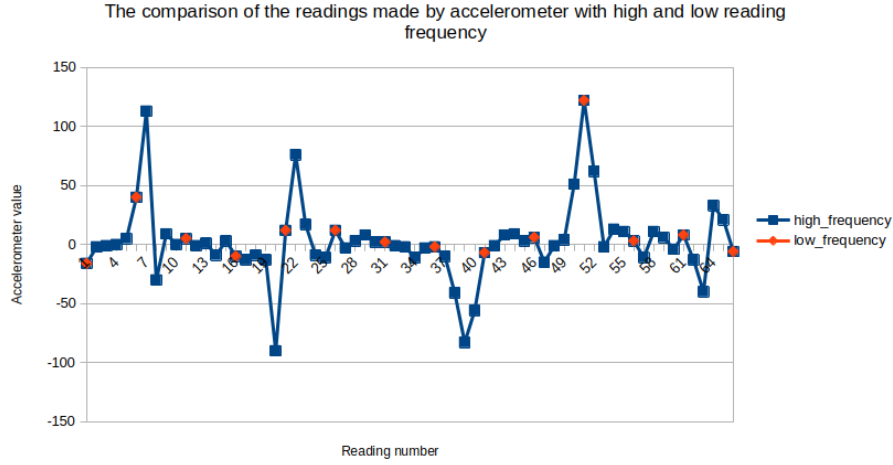


Figure 5: The low frequency of the readings caused only one out of the five triggers of the accelerometer to register properly. Misses like this would cause False Negative readings and the system would either not start up (lack automation) or not turn off (defeat the purpose of saving resources).

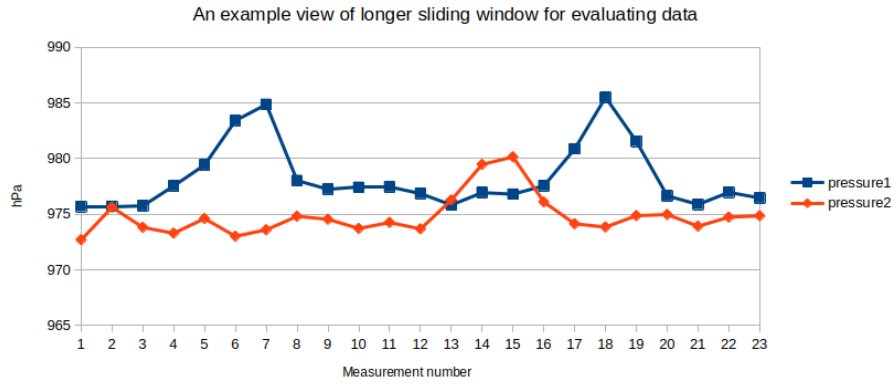


Figure 6: The longer interval and sliding window for evaluating the data caused a mistake where an event of pressure1 → pressure2 was evaluated as an event of pressure2 → pressure1, due to peaks of the measurements.

6 Discussion

Our project fulfilled its purpose almost completely — that is, it was a school project to try out technologies so that we can gain experience. The problem we decided we want to solve definitely has many other, mostly more simple, effective and elegant solutions. Here are drawbacks we recognize our project

has just to name a few:

1. **Entrance check:** In real application, entrance/presence of people would *not* have been checked by monitoring the door with sensors. We could assume, that someone who wishes for a smart home management has a smartphone. We could also assume that the phone would be either connected to owner's WiFi or would or send its current location with a trigger being it colliding with a zone created over the apartment (using GPX for example).
2. **Temperature control:** If the user has a smart AC unit to which we can send messages through an API, it probably already has its own application we could use. Creating our own solution would make sense if the AC unit was not smart and we needed to regulate everything by ourselves. This might include building a tool to manually adjust wall-mounted radiator settings. This was, however, outside of the project scope.
3. **Information gathering:** It would be more sensible to only have the temperature monitoring done in intervals. The rest, like the entrance check, should be done with tools that support events and listeners. In this way, we could reduce the amount of messages passed drastically, because only the relevant information would be sent out. Reducing unnecessary messages is best done closest to their source. This would also make it erase the need to optimize the interval at which we check people entering/leaving premises.
4. **Further Improvements:** When considering the steps that can be taken to improve this system in future iterations, one point that stands out is user management and configuring the system to store information on multiple users such as their own optimal temperatures and change the temperature according to the person in the home. Also, other smart home appliances such as lighting can be integrated into this system as well. Finally, another direction that be looked into is the creation of a low-power mode in order to reduce the energy consumption of the system when not in use.

7 Appendix

```
1 from machine import Pin
2 import network
3 import utime
4 import ssl
5 from bmp280 import BMP280
6 from umqtt.simple import MQTTClient
7 import connect_to_wifi
8 import config
9
10 connect_to_wifi.connect()
11
12 # Imports necessary libraries, connects and initializes ISM330DLC.
13 # Use read_acceleration(), init_ism330dlc().
14 import acc2
15 acc2.init_ism330dlc()
16
17 i2c = machine.I2C(1, sda=Pin(26), scl=Pin(27), freq=100000)
18 utime.sleep_ms(100)
19 bmp1 = BMP280(i2c)
20
21 led_pin = Pin('LED', Pin.OUT)
22
23 context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
24 context.verify_mode = ssl.CERT_NONE
25
26 def connectMQTT():
27     client = MQTTClient(client_id=b"syed",
28                         server=config.MQTT_BROKER,
29                         port=8883,
30                         user=config.MQTT_USER,
31                         password=config.MQTT_PWD,
32                         keepalive=7200,
33                         ssl=context)
34     client.connect()
35     return client
36
37 client = connectMQTT()
38
39 def publish(topic, value):
40     print(topic)
41     print(value)
42     client.publish(topic, value)
43     print("publish Done")
44
45 people_inside = 0
46
47 def led_blink(ms:int):
48     led_pin.on()
49     utime.sleep_ms(ms)
50     led_pin.off()
51     utime.sleep_ms(100)
52
53 def on_message(topic, msg):
54     print(f"Received message: {msg} on topic: {topic}")
55
```

```

56     if topic == b"picow/entrance":
57         if msg == b"Enter":
58             led_blink(1000)
59         elif message == b"Leave":
60             pass
61
62     #     if topic == b"picow/temp_control":
63     #         if msg == b"up":
64     #             led_blink(100)
65     #             led_blink(100)
66     #         elif msg == b"down":
67     #             led_blink(100)
68     #             led_blink(100)
69     #             led_blink(100)
70
71 client.set_callback(on_message)
72 client.subscribe(b"picow/entrance")
73 client.subscribe(b"picow/temp_control")
74
75 while True:
76     # Read sensor data
77     temperature = bmp1.temperature
78     pressure = bmp1.pressure/100
79     #humidity = sensor_reading.values[2]
80     acc_x = acc2.read_acceleration()/100
81
82     print(f"Acceleration: X={acc_x}")
83     print(f"Temperature: {temperature}C")
84     print(f"Pressure: {pressure}hPa")
85
86     # Publish as MQTT payload
87     publish('picow/temperature', f"{temperature:.2f}")
88     publish('picow/pressure1', f"{pressure:.2f}")
89     publish('picow/accelerometer', f"{acc_x}")
90     client.check_msg()
91
92     # Delay 2 seconds
93     utime.sleep(2)

```

Listing 1: Main Raspberry Pi code

```

1 import network
2 import socket
3 import utime
4 import config
5
6 def connect():
7     ssid = config.ssid
8     password = config.pwd
9
10    wlan = network.WLAN(network.STA_IF)
11    wlan.active(True)
12    wlan.connect(ssid, password)
13
14    max_wait = 10
15    while max_wait > 0:
16        if wlan.status() < 0 or wlan.status() >= 3:
17            break

```

```

18     max_wait -= 1
19     print('waiting for connection...')
20     utime.sleep(1)
21
22     # Handle connection error
23     if wlan.status() != 3:
24         raise RuntimeError('wifi connection failed')
25     else:
26         print('connected')
27         status = wlan.ifconfig()
28         print('ip = ' + status[0])

```

Listing 2: Code to connect to WiFi

```

1  from machine import Pin, I2C
2  import time
3
4  _i2c = I2C(1, sda=Pin(26), scl=Pin(27), freq=400000)
5  _ISM330DLC_ADDR = 0x6A # Default address for the ISM330DLC
6
7  # ISM330DLC register addresses
8  _WHO_AM_I = 0x0F
9  _CTRL1_XL = 0x10 # Control register for accelerometer
10 _OUTX_L_XL = 0x28 # X-axis acceleration (low byte)
11 _OUTX_H_XL = 0x29 # X-axis acceleration (high byte)
12
13 def init_ism330dlc():
14     # Check if the device is connected
15     who_am_i = _i2c.readfrom_mem(_ISM330DLC_ADDR, _WHO_AM_I, 1)[0]
16     if who_am_i == 0x6B:
17         print("ISM330DLC detected!")
18     else:
19         print(f"Device not found! {who_am_i}")
20         return False
21
22     # Configure accelerometer (e.g., enable accelerometer at 104 Hz
23     )
24     _i2c.writeto_mem(_ISM330DLC_ADDR, _CTRL1_XL, bytes([0x40]))
25     return True
26
27 def read_acceleration():
28     # Read X-axis acceleration
29     x_l = _i2c.readfrom_mem(_ISM330DLC_ADDR, _OUTX_L_XL, 1)[0]
30     x_h = _i2c.readfrom_mem(_ISM330DLC_ADDR, _OUTX_H_XL, 1)[0]
31     x = (x_h << 8) | x_l
32     # Convert to signed 16-bit
33     if x > 32767:
34         x -= 65536
35     return x

```

Listing 3: Code responsible for initializing and use of the accelerometer

```

1  ssid = '<network name>'
2  pwd = '<password>'
3
4  MQTT_BROKER_STR = '<token>.s1.eu.hivemq.cloud'
5  MQTT_BROKER = b'<token>.s1.eu.hivemq.cloud'

```

```

6 MQTT_PORT= 8883
7 MQTT_USER = b'<username>'
8 MQTT_PWD = b'<password>'

```

Listing 4: The config file

```

1 from machine import Pin, I2C
2 import network
3 import time
4 from bme280 import BME280
5 from umqtt.robust import MQTTClient
6 import ssl
7 import config
8
9 led_pin = Pin('LED', Pin.OUT)
10
11 # Setup Wi-Fi
12 ssid = config.ssid
13 password = config.pwd
14
15 # Connect to Wi-Fi
16 wlan = network.WLAN(network.STA_IF)
17 wlan.active(True)
18 wlan.connect(ssid, password)
19
20 connection_timeout = 10
21 while connection_timeout > 0:
22     if wlan.status() == 3: # Connected
23         break
24     connection_timeout -= 1
25     print('Waiting for Wi-Fi connection...')
26     time.sleep(1)
27
28 # Check if connection was successful
29 if wlan.status() != 3:
30     led_pin.on()
31     time.sleep_ms(1000)
32     led_pin.off()
33     raise RuntimeError('[ERROR] Failed to establish a network connection')
34 else:
35     led_pin.on()
36     time.sleep_ms(500)
37     led_pin.off()
38     time.sleep_ms(500)
39     led_pin.on()
40     time.sleep_ms(500)
41     led_pin.off()
42     print('[INFO] CONNECTED!')
43     network_info = wlan.ifconfig()
44     print('[INFO] IP address:', network_info[0])
45
46 # Config SSL connection with Transport Layer Security encryption (
47     no cert)
48 context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT) # Connect as
49     client, not server/broker
50 context.verify_mode = ssl.CERT_NONE # Do not verify server/broker
51     cert

```

```

49
50 # MQTT client connect
51 client = MQTTClient(
52     client_id=b'picow2',
53     server=config.MQTT_BROKER,
54     port=config.MQTT_PORT,
55     user=config.MQTT_USER,
56     password=config.MQTT_PWD,
57     ssl=context,
58     keepalive=300 # Keep-alive set to 5 minutes
59 )
60 client.connect()
61
62 # Unified callback function
63 def on_message(topic, msg):
64     print(f"Received message: {msg} on topic: {topic}")
65     if msg == b"up":
66         led_pin.on()
67         time.sleep_ms(500)
68         led_pin.off()
69         time.sleep_ms(500)
70         led_pin.on()
71         time.sleep_ms(500)
72         led_pin.off()
73         print("Optimal temperature to move up")
74     elif msg == b"down":
75         led_pin.on()
76         time.sleep_ms(500)
77         led_pin.off()
78         time.sleep_ms(500)
79         led_pin.on()
80         time.sleep_ms(500)
81         led_pin.off()
82         time.sleep_ms(500)
83         led_pin.on()
84         time.sleep_ms(500)
85         led_pin.off()
86         print("Optimal temperature to move down")
87
88 client.set_callback(on_message)
89 client.subscribe(b"picow/temp_control")
90
91 # Main loop with reconnection and keep-alive
92 while True:
93     try:
94         # Check for incoming messages
95         client.check_msg()
96
97         # Optional: Publish a keep-alive message every 60 seconds
98         if time.ticks_ms() % 60000 == 0:
99             publish(client, b"picow/keepalive", b"ping")
100
101     except OSError as e:
102         print(f"[ERROR] MQTT connection lost: {e}. Attempting to
reconnect...")
103         time.sleep(5) # Wait before attempting to reconnect
104         try:

```

```

105         client.connect() # Reconnect to the broker
106         client.subscribe(b"picow/control") # Resubscribe to
    topics
107         print("[INFO] Reconnected to MQTT broker.")
108         except Exception as e:
109             print(f"[ERROR] Reconnection failed: {e}")
110
111     # Sleep to control the loop frequency
112     time.sleep(2)

```

Listing 5: The temperature code for raspberry pi

```

1 import React, { useEffect, useState } from 'react';
2 import { View, Text, Button, StyleSheet,
3     SafeAreaView,
4     TouchableOpacity,
5     ScrollView,
6     Table, Row, TableWrapper, Rows } from 'react-native';
7 import { LineChart } from 'react-native-chart-kit';
8 import { Dimensions } from 'react-native';
9 import { fetchDashboardData } from '../api/data';
10 import client from '../mqttClient';
11 import moment from 'moment-timezone';
12 import CustomScreenHeader from './CustomScreenHeader';
13 import {
14     responsiveScreenHeight as hp,
15     responsiveScreenWidth as wp,
16     responsiveScreenFontSize as RF,
17 } from 'react-native-responsive-dimensions';
18
19 export default function DashboardScreen({ navigation }) {
20     const [data, setData] = useState({ temperature: 0, pressure: 0 });
21     const [mqttTemperature, setMqttTemperature] = useState(null);
22     const [mqttTemperatureGrp, setMqttTemperatureGrp] = useState([0]);
23     const [mqttPressure, setMqttPressure] = useState(null);
24     const [indicate, setIndicate] = useState(true);
25     const [mqttEntrance, setMqttEntrance] = useState(null);
26     const [mqttTempControl, setMqttTempControl] = useState(null);
27
28     const [showHeader, setShowHeader] = useState(true);
29     const [showTempContrl, setShowTempContrl] = useState(true);
30     //table
31     const [mqttData, setMqttData] = useState([]);
32
33     const labels = Array.from({ length: 6 }, (_, index) =>
34         moment().add(index, 'seconds').format('HH:mm:ss')
35     );
36
37     const myArray = [
38         ["Ford1", "BMW1", "Fiat1"],
39         ["Ford2", "BMW2", "Fiat2"],
40         ["Ford3", "BMW3", "Fiat3"]
41     ];
42
43     useEffect(() => {

```

```

45
46   setTimeout(toggleHeader,9000);
47   setTimeout(toggleTempControl,9000);
48
49   // Subscribe to MQTT topics
50   client.subscribe('picow/temperature');
51   client.subscribe('picow/entrance');
52   client.subscribe('picow/temp_control');
53
54   client.on('message', (topic, message) => {
55
56       const payload = message.toString();
57       const cleanedPayload = payload.replace(/C/g, '');
58       const payloadGrp = parseFloat(cleanedPayload);
59
60
61
62       if (topic === 'picow/temperature') {
63           setMqttTemperature(cleanedPayload);
64
65           if (!isNaN(payloadGrp)) { // Check if the parsed value
is a valid number
66               setMqttTemperatureGrp((prev) => [...prev.slice(-9),
payloadGrp]);
67           } else {
68               console.error('Invalid temperature value:',
cleanedPayload);
69           }
70       }
71
72       const timestamp = new Date().toLocaleTimeString();
73       console.log("timestamp :", timestamp);
74
75       if (topic === 'picow/entrance') {
76
77           setMqttEntrance(payload);
78           setMqttData({ topic, value: payload, time: timestamp })
79       };
80       setMqttData((prev) => [
81           ...prev,
82           { topic, value: payload, time: timestamp },
83       ]);
84
85       if (topic === 'picow/temp_control') {
86           setMqttTempControl(payload);
87       }
88   });
89
90
91   // Cleanup
92   return () => {
93       client.unsubscribe('picow/temperature');
94   };
95
96   }, []);
97

```



```

98   const toggleHeader = () => setShowHeader(showHeader => !
    showHeader);
99   const toggleTempControl = () => setShowTempContrl(showTempContrl
    => !showTempContrl);
100
101
102   const tableHead = ['Time', 'Topic', 'Value'];
103   const tableData = mqttData.map(({ time, topic, value }) => [
104     time,
105     topic,
106     value,
107   ]);
108
109
110   // Function to publish data to the MQTT topic
111   const handlePublishTempUp = () => {
112     client.publish('picow/temp_control', "up", { qos: 1 }, (error)
    => {
113       if (error) {
114         console.error('Error publishing message:', error);
115       } else {
116         console.log('Message published successfully:', "up");
117       }
118     });
119   };
120   const handlePublishTempDown = () => {
121     client.publish('picow/temp_control', "down", { qos: 1 }, (error)
    => {
122       if (error) {
123         console.error('Error publishing message:', error);
124       } else {
125         console.log('Message published successfully:', "down");
126       }
127     });
128   };
129
130
131   return (
132
133     <SafeAreaView style={styles.container}>
134       <CustomScreenHeader />
135
136       <View style={styles.secondSection}>
137
138         {showTempContrl && <View style={styles.heatingLabel}>
139           <Text style={styles.heatingLabelText}>{mqttTempControl
    === "up" ? "Heating in Progress" : "Cooling Down"}</Text>
140         </View>}
141
142
143       <View style={styles.infomationSection}>
144
145         <Text style={styles.currentTemp}>Current Temperature: </
    Text>
146         <Text style={styles.currentValue}> {mqttTemperature || '
    Loading...'} C </Text>
147

```

```

148     </View>
149
150     <View>
151
152         <LineChart
153             data={{
154                 labels: labels,
155                 datasets: [
156                     {
157                         data: mqttTemperatureGrp
158                     }
159                 ]
160             }}
161             width={Dimensions.get("window").width - 40} // from
react-native
162             height={220}
163             // yAxisLabel="$"
164             yAxisSuffix=" C "
165             // yAxisInterval={1} // optional, defaults to 1
166             chartConfig={{
167                 backgroundColor: "#e26a00",
168                 backgroundGradientFrom: "#fb8c00",
169                 backgroundGradientTo: "#ffa726",
170                 decimalPlaces: 2, // optional, defaults to 2dp
171                 color: (opacity = 1) => 'rgba(255, 255, 255, ${
opacity})',
172                 labelColor: (opacity = 1) => 'rgba(255, 255, 255, $
{opacity})',
173             }}
174             // bezier
175             style={{
176                 marginVertical: 8,
177                 borderRadius: 16
178             }}
179         />
180     </View>
181
182     <View style={styles.heatingLabelContorl}>
183         <Text style={styles.heatingLabelText}>Temperature Control
</Text>
184     </View>
185
186     <TouchableOpacity
187         style={styles.sendSignal}
188         onPress={() => handlePublishTempUp}>
189         <Text style={styles.infoText}> Up</Text>
190     </TouchableOpacity>
191     <TouchableOpacity
192         style={styles.sendSignal}
193         onPress={() => handlePublishTempDown}>
194         <Text style={styles.infoText}> Down</Text>
195     </TouchableOpacity>
196
197 </View>
198 </SafeAreaView>
199
200 );

```

```

201 }
202
203 const chartConfig = {
204   backgroundGradientFrom: "#1E2923",
205   backgroundGradientFromOpacity: 0,
206   backgroundGradientTo: "#08130D",
207   backgroundGradientToOpacity: 0.5,
208   color: (opacity = 1) => `rgba(26, 255, 146, ${opacity})`,
209   strokeWidth: 2, // optional, default 3
210   barPercentage: 0.5,
211   useShadowColorFromDataset: false // optional
212 };
213
214 const styles = StyleSheet.create({
215   container: {
216     flex: 1,
217     // backgroundColor: '#a0cbe0',
218     alignItems: 'center',
219     overflow: 'auto'
220   },
221   secondSection: {
222     alignItems: 'center',
223     justifyContent: 'center',
224     marginTop: wp(10)
225   },
226   heatingLabel: {
227     backgroundColor: '#DA9441',
228     padding: wp(3),
229     borderRadius: 6,
230   },
231   heatingLabelText: {
232     fontSize: RF(2),
233     fontWeight: '700',
234     color: 'ffffff',
235   },
236   heatingLabelContorl: {
237     backgroundColor: '#DA9441',
238     padding: wp(3),
239     borderRadius: 6,
240     marginTop: wp(5)
241   },
242   infomationSection: {
243     padding: wp(5),
244     alignItems: 'center'
245   },
246   currentTemp:{
247     paddingTop: wp(5),
248     fontSize: RF(1.6),
249     fontWeight: '700',
250     color: '#000000',
251   },
252   currentValue:{
253     paddingTop: wp(3),
254     paddingBottom: wp(5),
255     fontSize: RF(4.5),
256     fontWeight: '800',
257     color: "#0570C0",

```

```

258 },
259 tempInfo: {
260   backgroundColor: '#DA9441',
261   marginTop: wp(3),
262   marginBottom: wp(4),
263   padding: wp(3),
264   width: wp(80),
265   borderRadius: 10
266 },
267 sendSignal: {
268   backgroundColor: '#3DAB4F',
269   padding: wp(3),
270   width: wp(80),
271   borderRadius: 10,
272   marginTop: wp(5)
273 },
274 infoText:{
275   textAlign:'center',
276   fontSize: RF(1.8),
277   color:'#ffffff',
278   fontWeight:'600'
279 },
280 containertb: {
281   flexGrow: 1,
282   padding: 16,
283   backgroundColor: '#f5f5f5',
284 },
285 title: {
286   fontSize: 18,
287   fontWeight: 'bold',
288   textAlign: 'center',
289   marginBottom: 16,
290 },
291 head: {
292   height: 40,
293   backgroundColor: '#f1f8ff',
294 },
295 wrapper: {
296   flexDirection: 'row',
297 },
298 text: {
299   margin: 6,
300   textAlign: 'center',
301 },
302 });

```

Listing 6: Dashboard of the mobile app

```

1 import React, { useState } from 'react';
2 import { View, Text, TextInput, Button, StyleSheet, SafeAreaView,
   TouchableOpacity, Image } from 'react-native';
3 import { login } from '../api/auth';
4 import {
5   responsiveScreenHeight as hp,
6   responsiveScreenWidth as wp,
7   responsiveScreenFontSize as RF,
8 } from 'react-native-responsive-dimensions';
9

```

```

10 export default function LoginScreen({ navigation }) {
11   const [username, setUsername] = useState('');
12   const [password, setPassword] = useState('');
13
14   const handleLogin = async () => {
15     navigation.navigate('Dashboard');
16   };
17
18   return (
19
20
21
22     <SafeAreaView style={styles.container}>
23       <View style={styles.logoSection}>
24         <Image
25           source={require('./image/logo.png')}
26           style={styles.imageView}
27         />
28       </View>
29
30       <View style={styles.inputSection}>
31
32         <TextInput style={styles.inputStyles} placeholder="Your
33         Name" />
34         <TextInput style={styles.inputStyles} placeholder="
35         Password" secureTextEntry={true} />
36
37       </View>
38       <View style={styles.buttonSection}>
39         <TouchableOpacity style={styles.loginButton} onPress={
40         handleLogin}>
41           <Text style={styles.buttonText}>Login</Text>
42         </TouchableOpacity>
43       </View>
44     </SafeAreaView>
45   );
46 }
47
48 const styles = StyleSheet.create({
49
50   container: {
51     flex: 1,
52     backgroundColor: 'ffffff',
53   },
54   imageView: {
55     width: wp(40),
56     height: wp(40),
57   },
58   logoSection: {
59     marginTop: wp(30),
60     alignItems: 'center',
61     justifyContent: 'center',
62   },
63   inputSection: {
64     marginTop: wp(15),
65     alignItems: 'center',
66     alignItems: 'center',

```

```

64     justifyContent: 'center',
65   },
66   inputStyles: {
67     width: wp(80),
68     padding: wp(4),
69     borderWidth: 0.3,
70     marginBottom: wp(4),
71     borderRadius: 5,
72     borderColor: '#4DABE9',
73   },
74   buttonSection: {
75     marginTop: wp(10),
76     alignItems: 'center',
77     justifyContent: 'center',
78   },
79   loginButton: {
80     width: wp(80),
81     backgroundColor: '#4DABE9',
82     borderRadius: 5,
83   },
84   buttonText: {
85     textAlign: 'center',
86     padding: wp(4),
87     fontSize: RF(2),
88     color: 'ffffff',
89     fontWeight: '700',
90   }
91 });

```

Listing 7: Login Screen of the mobile app

```

1  import React from 'react';
2  import {
3    View,
4    Text,
5    SafeAreaView,
6    StyleSheet,
7  } from 'react-native';
8  import {
9    responsiveScreenHeight as hp,
10    responsiveScreenWidth as wp,
11    responsiveScreenFontSize as RF,
12  } from 'react-native-responsive-dimensions';
13
14
15  const CustomScreenHeader = (props) => {
16
17    return (
18      <SafeAreaView style={styles.container}>
19        <View style={styles.subContainer}>
20          <Text style={styles.headertext}>Dashboard</Text>
21        </View>
22      </SafeAreaView>
23    );
24  };
25
26  export default CustomScreenHeader;
27

```

```

28 const styles = StyleSheet.create({
29   container: {
30     height: wp(13),
31     width: wp(90),
32     alignItems: 'center',
33   },
34
35   subContainer: {
36     flexDirection: 'row',
37     justifyContent: 'space-between',
38     alignItems: 'center',
39   },
40   headertext: {
41     color: '#000000',
42     fontSize: RF(3),
43     fontWeight: '700',
44     marginLeft: wp(3),
45   },
46 });

```

Listing 8: Custom Screen Header of the mobile app